

ClawBot 基础功能优化与整改方案

代码审查结论、实施路线与验收标准

项目	分支	评审日期	文档状态
co-WechatAgent	co-luanxu	2026-07-26	建议按优先级实施

执行摘要

当前项目的基础调用链已经可以运行，现有 Maven 测试共 16 项并全部通过；但在多用户隔离、网页访问安全、复合任务路由、长消息发送和异常可观测性方面仍存在明显风险。若直接扩大用户量或开放更多工具，问题会从“偶发回答不准确”升级为“跨用户上下文污染、内网探测、消息丢失或故障不可定位”。

整改结论

优先完成 4 个 P0/P1 项：按用户隔离会话、封堵网页 SSRF、让责任链支持复合任务、统一长文本分片。之后再收紧新闻识别、接入登录超时与统一告警。

目标结果

- 不同微信用户之间的历史、摘要、去重集合和清空操作完全隔离。
- 所有外部 URL 访问经过统一安全策略，不可访问本机、内网、链路本地地址或超大响应体。
- 一条消息中的多个独立需求能被拆分、执行并聚合回复，不被第一个处理器提前截断。
- 所有文本回复统一按微信长度限制分片；任何异常都有本地完整日志和钉钉摘要通知。

1. 风险优先级总览

下表按影响范围、被触发概率和修复收益综合排序。P0 表示应在继续扩展工具前完成；P1 表示进入稳定版本前完成；P2 可与体验优化并行推进。

级别	问题	主要影响	核心改法
P0	会话状态未按用户隔离	历史串话、清空影响所有人、并发不安全	引入 userId → ConversationMemory
P0	网页提取存在 SSRF 与内存风险	可能访问内网/本机，超大响应导致内存压力	统一 UrlAccessPolicy + 大小/重定向限制
P1	责任链首个命中即停止	一条消息多个需求只能完成部分	返回 handled/continue/remainingText
P1	文本回复未统一分片	聚合答案过长，发送失败或截断	抽取 WeChatMessageSender
P1	异常被吞且用户看到原始错误	难排障、可能泄漏内部细节	集中 ErrorReporter + 安全提示
P1	登录超时配置未生效	等待循环可能永久阻塞	实现 deadline、关闭和重试
P2	新闻关键词过宽	误触发检索、重复上下文、成本增加	优先 Function Calling 或收窄意图
P2	配置与依赖存在漂移	假配置、版本不一致、启动负担	清理默认值、Java 版本和依赖

2. 当前处理流程的关键症结

现状流程

微信消息 → 基础处理器按优先级判断 → 第一个命中的处理器直接结束 → 文本处理器调用规划/模型/工具 → 单次发送回复

这条链路适合“一个输入对应一个动作”的场景，但不适合复合消息。优先级本身没有问题，问题在于处理器只表达“是否处理”，不能表达“已处理一部分、是否继续、还剩哪些文本”。多任务规划即使能拆分，在进入专用处理器之前也可能被责任链提前截断。

3. 设计原则

- 用户隔离优先于模型能力：上下文正确性必须由应用层保证。
- 工具调用与网络访问默认不可信：所有 URL、参数、响应和超时均需边界。
- 编排与执行分离：规划器只拆任务，执行器只运行任务，发送器只负责渠道限制。

- 错误对用户友好、对开发者完整：两类信息分层输出。

4. 高优先级整改项

4.1 会话记忆按用户隔离

问题：TextMessageHandler 作为单例持有 longTermSummary、recentMessages 和 processedMsgIds。多个微信用户会共享这些字段；任意用户执行清空操作会影响所有人，而且普通集合在并发回调下不安全。

建议结构：

- ConversationMemory：保存摘要、最近消息、更新时间和清理锁。
- ConversationMemoryStore：按稳定的 userId/openId 获取、清空、过期淘汰。
- MessageDeduplicationService：独立维护消息 ID 的 TTL 有界缓存。
- 持久化文件名包含不可逆用户标识摘要，避免明文暴露并防止用户间覆盖。

验收点

A 用户的对话不会出现在 B 用户提示词中；A 清空记忆不影响 B；并发 100 条消息无集合异常；缓存能按 TTL 自动淘汰。

4.2 网页访问安全与资源边界

问题：网页提取允许访问 localhost、私网、链路本地地址，并使用不受限的字节数组接收响应。若模型或用户提供恶意 URL，可能形成 SSRF；若响应体过大，会增加内存占用。

统一 UrlAccessPolicy 应至少包含：

- 只允许 http/https；拒绝 localhost、环回、私网、链路本地、保留地址和非常规端口。
- 解析 DNS 后校验所有 IP；每次重定向重新解析并校验，限制最多 3 次。
- 设置连接、读取和总超时；正文上限建议 2 MB，超限立即中止。
- Content-Type 白名单；日志记录目标域名、耗时和拒绝原因，但不记录敏感查询参数。

复用要求

新闻抓取、网页摘要及未来所有联网工具必须共享同一套 URL 访问策略，禁止各自复制校验逻辑。

4.3 责任链支持复合任务

将处理器返回值由简单布尔值改为结构化结果，例如 `HandlerResult(handled, continueChain, remainingText, outputs)`。专用处理器只消费自己能处理的片段，剩余文本继续进入后续处理器或统一任务执行器。

推荐流程：

1. 先做轻量输入归一化和安全检查。
2. 规划器输出 Task 列表，每项包含类型、原始文本、依赖和顺序。
3. 无依赖任务可并行；有依赖任务串行，并设置单任务超时。
4. 按原问题顺序聚合结果，再交给统一发送器分片。

5. 稳定性与可观测性整改

5.1 统一微信消息发送

目前文档处理存在约 1500 字分片逻辑，而普通文本与多任务聚合结果可能一次发送。应抽取 `WeChatMessageSender`，统一完成文本清洗、分片、顺序发送、失败重试和发送结果记录。

- 优先按段落边界切分，其次按句号，最后按硬长度切分。
- 每段保留任务编号和上下文；避免把 Markdown 代码块拆成无效片段。
- 单段失败时只重试该段，并使用幂等键防止重复回复。

5.2 统一异常报告

处理器不应静默吞异常，也不应直接把 `e.getMessage()` 原样发给用户。建议引入 `ErrorReporter`：本地日志记录完整堆栈与关联 ID；钉钉发送脱敏摘要；微信只返回可理解的提示和关联 ID。

异常类型	本地日志	钉钉通知	用户提示
模型/API 超时	记录模型、耗时、任务类型	发送告警摘要	“服务响应超时，请稍后重试”
工具参数错误	记录校验结果	达到阈值后告警	指出需补充的字段

异常类型	本地日志	钉钉通知	用户提示
登录/会话失效	记录状态转换	立即告警并附登录入口	提示管理员处理
未知异常	完整堆栈 + correlationId	立即告警	通用失败提示 + 关联 ID

5.3 登录生命周期

wechat.login.timeout-ms 已有配置入口但等待循环未使用，可能永久阻塞。登录流程应维护明确状态：INIT → QR_READY → WAITING → LOGGED_IN / TIMEOUT / FAILED，并在 deadline 到达后关闭资源、通知钉钉、按退避策略重试。

5.4 新闻识别与多任务上下文

“今天、昨天、最近、有什么、科技”等词过于宽泛，会把普通问题误判为新闻并提前检索；多任务时还可能把同一新闻上下文复制到每个任务。建议取消宽关键词预取，优先让模型通过 Function Calling 明确选择新闻工具；若保留规则，应要求主题词与时效词同时出现，并只把检索结果绑定到对应任务。

5.5 性能与成本

多任务规划器不必对每条消息都调用一次模型。可先用轻量门控判断是否包含并列连接词、多个问号或多个已知工具意图；单任务直接走现有链路，复杂输入才进入规划模型。对等价短输入可使用短期规划缓存。

6. 目标架构

建议调用链

WeChat Adapter → Input Guard → Task Planner → Task Executor → Tool Registry / LLM → Result Aggregator → WeChatMessageSender

横切能力由独立组件提供：ConversationMemoryStore 负责用户上下文；UrlAccessPolicy 约束联网工具；ErrorReporter 统一日志和钉钉通知；ConfigurationProperties 承载模型与第三方 API 配置。

组件职责

组件	职责	边界
WeChat Adapter	接收/确认消息，提取 userId、msgId、文本和附件	不包含模型或业务判断

组件	职责	边界
Input Guard	去重、限流、输入长度和基础安全校验	快速、无模型调用
Task Planner	把复合消息拆为有序任务与依赖	只规划，不执行
Task Executor	调度工具/LLM，控制并发、超时和取消	输出结构化结果
Tool Registry	注册工具 schema 与执行器	工具配置外置
Result Aggregator	按原问题顺序组合成功与失败结果	保留部分成功
Message Sender	渠道分片、重试、幂等和发送日志	统一所有回复出口

7. 分阶段实施路线

阶段	目标	交付物	退出条件
阶段 1	数据正确性	会话隔离、有界去重缓存	跨用户测试、并发测试通过
阶段 2	网络安全	UrlAccessPolicy、超时、大小与重定向限制	SSRF 用例全部拒绝
阶段 3	复合任务	HandlerResult、规划/执行/聚合拆分	3 类复合场景全部完成
阶段 4	渠道稳定	统一分片、重试、关联 ID	超长回复完整可达
阶段 5	运维体验	登录超时、钉钉告警、新闻规则收紧	生命周期与告警测试通过
阶段 6	清理收尾	配置、依赖和测试覆盖整理	构建无警告、配置清晰

并发策略

只并行执行彼此独立、无共享写入的任务；涉及会话状态、同一文件或具有顺序语义的任务保持串行。
每个任务必须有超时，总请求还需设置总体 deadline。

8. 配置与依赖清理

模型、第三方 API、超时和渠道限制应全部通过 @ConfigurationProperties 管理，并在启动时校验必填值。敏感信息只引用环境变量，禁止提交真实密钥。

配置/依赖	发现	建议
tianapi.api.key	当前可能使用占位默认值，容易被误判为已配置	默认空字符串；启用工具时 fail-fast
模型配置	供应商、模型名、温度、超时可能分散在服务类	集中为 llm.*，按用途区分 planner/chat/tool
wechat.login.timeout-ms	存在但未真正控制等待循环	绑定到登录 deadline 并覆盖测试
钉钉配置	应避免随分支或本地文件丢失	只提交变量占位；真实值放环境变量

配置/依赖	发现	建议
Java 版本	项目说明为 Java 21, POM 可能仍声明 17	统一编译、运行与 CI 为 21
MySQL 驱动	当前未见明确使用场景	无持久化需求则删除
Web Starter/8081	项目无 Controller 时可能是额外启动负担	确认 SDK 需要后保留, 否则设为非 Web
NotificationService	接口上的 @Component 冗余	仅实现类注册为 Bean

9. 回归测试矩阵

测试域	关键用例	期望结果
多用户会话	A/B 交替聊天、分别清空、并发发送	上下文完全隔离, 无并发异常
消息去重	重复 msgId、过期 msgId、超过容量	窗口内去重, 过期可处理, 容量受控
复合任务	天气+汇率、文档+问答、图片+普通文本	每项均执行, 结果按输入顺序聚合
URL 安全	127.0.0.1、私网域名、DNS 重绑定、重定向	全部拒绝并记录原因
响应边界	超时、2 MB 以上正文、非文本类型	及时中止, 不造成内存峰值
长消息	超过单段限制、多段落、代码块	完整分片、有序到达、无重复
告警	模型异常、登录超时、未知异常	本地堆栈完整, 钉钉脱敏, 用户提示安全
配置	缺少必填密钥、非法超时、生产环境启动	启动时明确失败或禁用对应工具

当前基线: Maven 测试共 16 项并全部通过。该结果说明现有路径未见立即回归, 但尚不足以覆盖上述路由、多用户、URL 安全、钉钉告警和登录生命周期场景。

10. 验收清单

FUNCTIONAL

- 一条消息包含 2-3 个独立需求时, 全部需求都有明确结果; 单项失败不吞掉其他成功结果。
- 汇率、天气、文档、图片、新闻等工具均从 Tool Registry 注册并通过统一执行流程调用。
- 所有回复从同一发送组件发出, 长文本可稳定分片。

SECURITY & RELIABILITY

- 任何联网工具均无法访问本机、私网、链路本地和保留地址。
- 所有外部调用都有连接/读取/总超时和响应大小限制。
- 会话、去重和缓存都具备用户边界、容量边界和时间边界。
- 敏感配置不入库; 日志和钉钉消息不包含 API Key、Webhook 签名或完整用户隐私。

OPERATIONS

- 登录成功、登录超时、会话失效和未捕获异常均产生符合等级的钉钉通知。
- 每次用户请求具备 correlationId，可贯穿规划、工具调用、聚合、发送和告警。
- CI 使用 Java 21，clean test 与 clean package 均通过。

11. 关键取舍

决策	收益	代价
先隔离会话，再扩工具	避免工具越多，跨用户污染面越大	需要调整 TextMessageHandler 状态管理
统一联网策略	一次修复覆盖现在和未来工具	初期需适配现有抓取代码
规划器按需调用	降低延迟和模型成本	门控规则需持续用真实消息校准
部分成功优先	复合任务中单项失败不影响整体	聚合结果需明确标注失败项

12. 完成定义

Definition of Done

上述 P0/P1 项完成；新增回归测试全部通过；配置说明与 application.properties 示例同步；无真实密钥进入 Git；在两个微信用户并发、三任务复合消息、超长回复和恶意 URL 四类验收场景下均表现稳定。

建议每个阶段独立提交并通过 PR/MR 合并，避免把架构调整、配置迁移和工具新增混在同一提交中，以便审查、回滚和责任归属。